



Shakti overview

RISE Lab

IITM

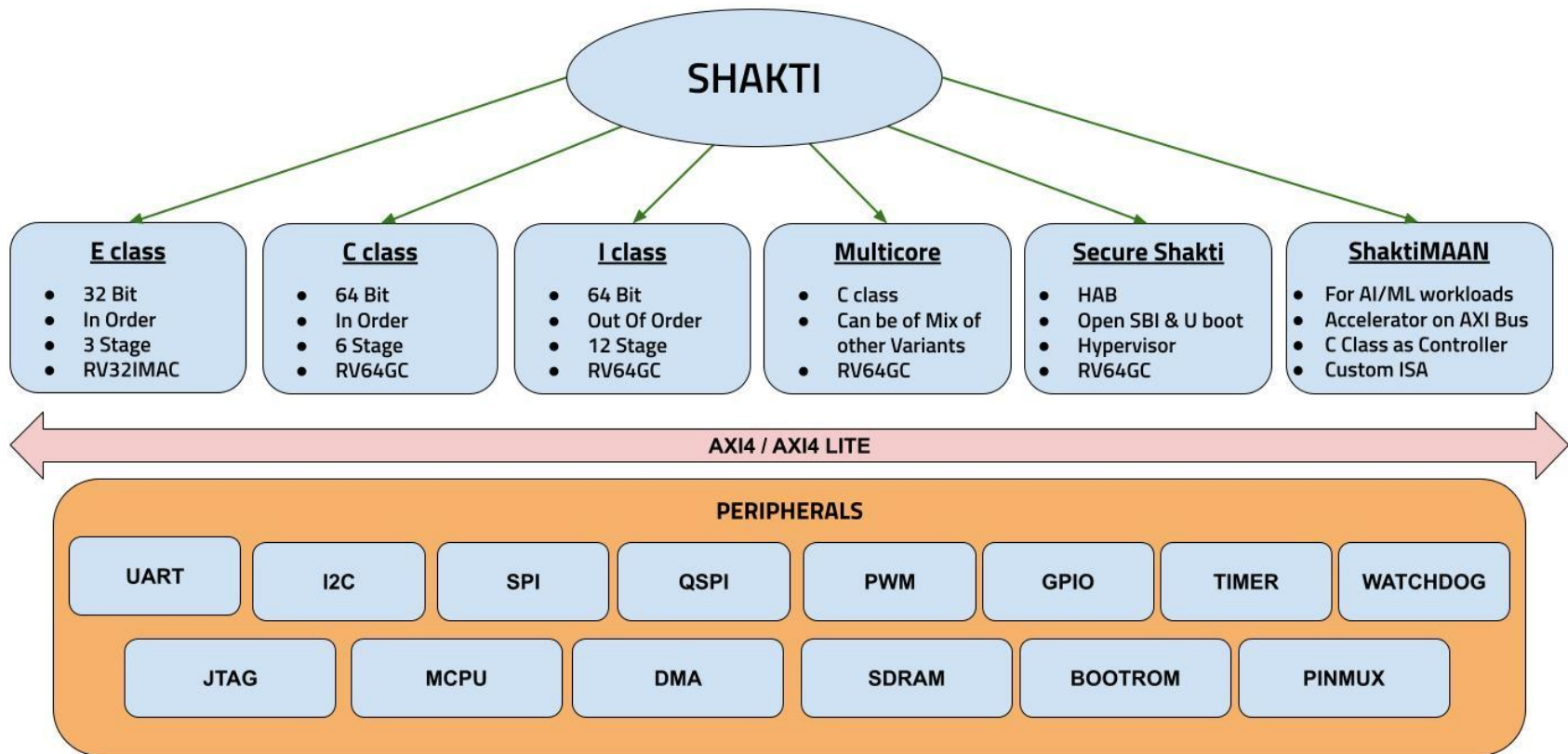
Webinar Schedule

1	GC2025 processors overview & Intro. to FPGAs	1 hour
2	GC2025 tool setup.	1 hour
3	Bit stream generation & programming.	1 hour
4	SoC overview.	1 hour
5	SoC register description	1 hour 30 mins
6	Porting Bluespec based IPs	1 hour
7	Porting Verilog based IPs with AXI interface	1 hour
8	Porting Verilog based IPs without AXI interface.	1 hour
9	Shakti SDK overview.	1 hour
10	Shakti SDK application code generation.	1 hour
11	Shakti SDK project generation	1 hour
12	Shakti SDK benchmarking	1 hour
13	Porting driver code to Shakti SDK (LCD).	1 hour
14	Linux porting to Shakti.	1 hour 30 mins
15	Linux driver porting for Shakti.	1 hour
16	Porting of Zephyr RTOS to Shakti.	1 hour 30 mins

Genesis of SHAKTI

- **SHAKTI** - family of open source RISC-V processors ranging from simple single core processors to server-grade multi-core processors.
- Today the **SHAKTI** initiative has grown into a comprehensive ecosystem that will provide industry-grade open source processors, AI/ML accelerators, communication fabrics, interconnects, peripherals and storage controllers.
- This initiative has resulted in 7 startups so far.

SHAKTI Ecosystem



E Class vs C Class vs I Class

E-Class

- ❖ Operating frequency: Upto 400 MHz
- ❖ Positioned against ARM's Cortex Mx class cores
- ❖ Target: IoT devices, Edge Devices, Robotic Control, Smart cards

C-Class

- ❖ Operating frequency: > 1+ GHz
- ❖ Positioned against ARM's Cortex A35/A55
- ❖ Target: Reliable Computing, Secure Computing, IoT & Edge Computing hubs, Auto/Aerospace/Industrial Controls

I-Class

- ❖ Operating frequency: 1-2 GHz
- ❖ Positioned against ARM's Cortex A76 (Work in Progress)
- ❖ Target: High Performance Compute, Mobile, Storage and Networking segments

GC2025 Shakti Based Configurable Systems



Yamuna

- 32 bit C class core
- 6 stage Pipeline
- In order core
- RV32IMAC
- 16kB \$I & 16kB\$D
- 256MB DDR
- 8kB boot ROM.
- 32 x GPIOs
- 4 x General Purpose Timers
- 2 x SPI controller.
- 3 x UART
- 6 x PWM
- 2 x I2C
- 1 x PLIC
- 1 x CLINT
- Pinmux support.
- On board xADC
- Arduino compatible pins
- FTDI based debugging.
- Ethernet Lite IP
- RTOS Booted.

Ganga

- 64 bit C class core
- 6 stage Pipeline
- In order core
- RV64IMAC
- 16kB \$I & 16kB\$D
- 256MB DDR
- 8kB boot ROM.
- 16 GPIOs
- 2x General Purpose Timers
- 2 x SPI controller.
- 2 x UART
- 1 x I2C
- 1 x PLIC
- 1 x CLINT
- Pinmux support.
- Arduino compatible pins
- FTDI based debugging.
- Ethernet Lite IP
- Linux Booted.

Applications



Yamuna Applications

- Cater to Low power and low speed applications.
 - Wireless sensor networks.
 - Smart card reader.
 - RFID tags.
 - Home automation.
 - Agriculture equipments
 - Robotic Control
 - Edge Devices.
 - Simple Data Acquisition Systems.
 - Personal tracking systems.
 - Vehicle Tracking systems
 - Motor control applications.
 - Light control systems.

Ganga Applications

- Cater to Low/Medium power, high performance and higher speed applications.
 - Industrial Automation.
 - Reliable computing
 - Edge computing
 - Secure computing
 - Automobile control
 - Aerospace applications
 - Critical Monitoring systems.
 - IoT Gateways
 - Image/Video processing.
 - Unmanned vehicle systems.
 - Speech Processing Systems.

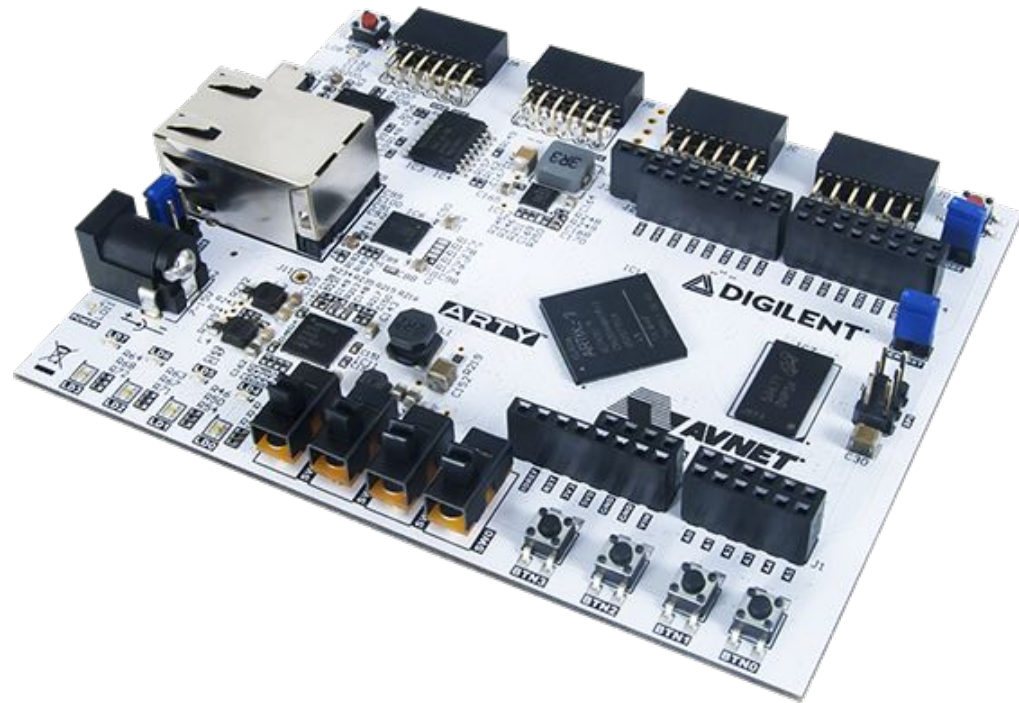
FPGA Based SoC Templates



Board Support

Shakti based SoCs are supported on Multiple Boards

- Nexys Video
- AC701
- VCU118
- Aldec
- proFPGA
- ZC702.
- Arty 100T & Arty 35T
- Genesys 2
- VC707



Shakti Peripherals



UART

- Configurable Baud rate
 - Upto 3Mbps.
- Complete UART support with RTS, CTS, DTR, DSR, CD,RI.
- Programmable character size.
- Programmable stop bits, parity bits.
- 16 byte FIFO.
- Read / Write options
 - Polling based
 - Interrupt based
- Integrated with ESP chip, 4G Modem, Lora & Zigbee.

I2C

- Supports Standard (100kHz) and Fast mode I2C (400kHz).
- Master Mode support.
- Configurable clock speed.
- 7 bit addressing.
- Supports
 - Polling based
 - Interrupt based
- FIFO based implementation as well available (32 Byte)

SHAKTI PERIPHERALS

SPI

- Standard SPI.
- Configurable clock speed (Tested upto 12.5MHz).
- Master and Slave mode support.
- Multi slave support
- Supports both 3 wire and 4 wire interface.
- Programmable clock polarity(CPOL) and phase (CPHA)
- 32 Byte FIFO depth.
- Programmable set up time and hold time.

PWM

- FPGA board Clock speed - 50MHz
- Configurable PWM
 - Clock divisor
 - Period
 - Duty Cycle
- Tested upto 7.5 MHz.
- Complementary Output support.
- Timer mode support.
- Programmable interrupts.

GPTimer

- 32 bit General Purpose Timer.
- Supports Up/Down/Up-down Counters.
- Also supports PWM.
- Captures the input.
- Continuous Counter mode.
- External reference clock can be used.
- Underflow/Overflow interrupts can be configured.

Watchdog Timer

- 32 bit Watchdog timer.
- Prevents the core from Stalling during code execution.
- Supports hard reset as well as interrupt mode.
- When the active bit is not triggered, the SoC will get reset on timeout.
- Watchdog timeout and external reset period can be configured in software.

GPIO

- General Purpose Input / Output.
- No of ports can be configurable (customizable).
- Separate SET, CLEAR & TOGGLE registers can be used to configure the individual port values.
- DATA register can be used to read the GPIO port values.
- DIRECTION register can be used to configure the GPIO port as input or output.
- Few GPIO pins can be configured for external interrupt.
- Interrupts can be configured either as active high or active low.
- Manys sensors verified and examples present in SDK.

SHAKTI PERIPHERALS

PLIC

- RISC-V compatible interrupt controller.
- Registers editable at bit level.
- Default and vector interrupts supported.
- Priority can be configured.
- Complete driver support with examples

CLINT

- Core Local Interrupt Timer.
- Machine Mode timer.
- Configure timer at tick level.
- MTIME & MTIMECMP registers.
- Interrupt mode supported.
- Complete driver support with examples.

PMP

- Allows physical memory access privileges to specified for each physical memory region.
- Can specify up to 64 regions
- Granularity can be as small as 4
- PMP checks by default applied to S and U memory/instruction access
- PMP checks may apply to M mode, if the regions are locked at reset, preventing even M mode SW to change it.

PinMux

- Used to configure and select the hardware pin configured for multiple functions based on the configured value.
- Pin mux register has to be configured to enable a peripheral function on a pin.
- Pin mux register is 32 bits.
- By default all pins are configured as GPIO.
- Different values can be configured in pin mux register to assign different functions to each and every pin.
- GPIO pins can be assigned to different peripheral devices
- Pinmux configuration can be increased for adding more functionality to each pin.

PinMux USED IN SP2020

PinMux Config Value	Bit Position															
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
00	GP10		GP9		GP6		GP5		GP3		GP2		GP1		GP0	
01	S2_CS		-		-		-		U2TX		U2RX		U1TX		U1RX	
10	PWM4		PWM3		PWM2		PWM1		PWM0		-		-		-	

PinMux Config Value	Bit Position															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
00	-		-		-		-		-		GP13		GP12		GP11	
01	-		-		-		-		-		S2CK		S2SO		S2SI	
10	-		-		-		-		-		-		-		PWM5	

Shakti Cryptographic Accelerators



AES

- Symmetric key encryption standard
- Support for multiple key lengths: 128b, 192b or 256b
- Support for various modes
 - ECB: Electronic Code Book
 - CBC: Cipher Block Chaining
 - CFB: Cipher FeedBack
 - OFB: Output FeedBack
 - CTR: Counter
- AES-256 Design
 - 16 unified SBoxes
 - AXI-4 slave
 - Buffers of depth 4 at input and output
 - Supports DMA with automatic flow control

SHA

- One-way function
- Typically used
 - To verify integrity of data
 - For digital signatures
- Various digest sizes -
 - 256 and 512
- A small change in input results in significant change in output.
- Engine Variants (SHA256) -
 - Sequential (64 clock cycles)
 - Sequential Unrolled (32 clock cycles)
 - Pipelined (64 stages)

FPGA - Field Programmable Gate Arrays



What is an FPGA?

- **Field Programmable Gate Array (FPGA)**
- **Reconfigurable Hardware** – FPGAs enable dynamic hardware customization.
- **HDL Programmability** – Designed using HDLs to model chip behavior.
- **Post-Deployment Adaptability** – Modify functionality even after deployment.

FPGA Technology Giants

AMD (Xilinx)^[1]



Intel (Altera)^[2]



Microchip (Microsemi)^[3]



[1] <https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga.html>

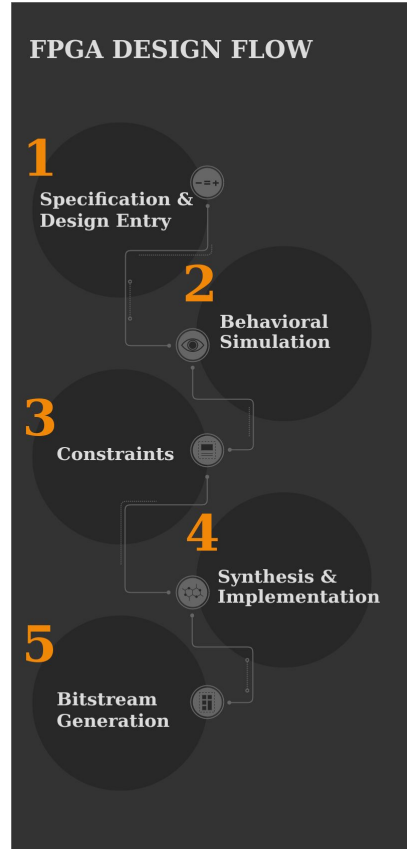
[2] <https://www.intel.com/content/www/us/en/products/details/fpga/aria.html>

[3] <https://www.microchip.com/en-us/products/fpgas-and-plds/fpgas>

Why FPGA

- ASIC - Application specific Integrated Circuit i.e. IC designed for a specific application.
- ASIC fabrication is a tapeout
- A costly process which involves millions of Dollars i.e. huge cost investment.
- Verification process is a must.
- Simulation & FPGA verification.
- Simulation - Will not guarantee on timing.
- FPGA - will provide both timing & functional validation i.e. at low frequencies.

FPGA Design Flow



FPGA Design Flow

- **Specification & Design Entry:** Getting the requirements/specifications & low/high level design.
- **Behavioral Simulation:** Writing the code (verilog, vhdl, system verilog & BSV).
- **Synthesis:** Converting the hdl to a netlist i.e. list of logic elements with connections along with optimization.
- **Simulation:** verification of the specification with simple test benches.

FPGA Design Flow

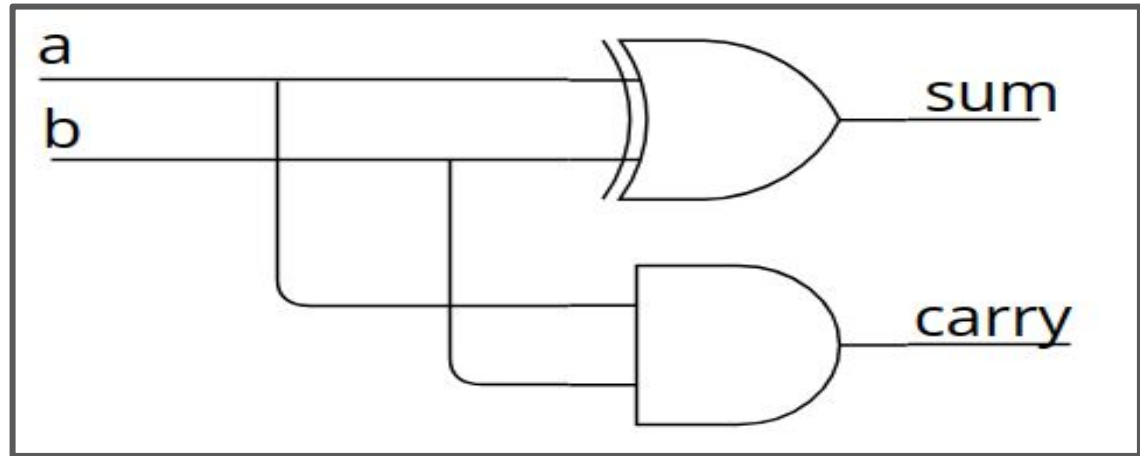
- **Implementation:** Process of converting the netlists to FPGA specific pattern
 - **Translate** - collects and merge netlists into a single net list & verify the constraints.
 - **Map:** Mapping the resources with net lists
 - **Place & Route:** physical placement of the netlist to the FPGA physical resources
- **Bitstream generation:** converting the implementation design into a FPGA understandable format.



A Step-by-Step Walkthrough in Nexys Video

1. Design Entry - HDL Coding (Verilog or VHDL or System Verilog)

```
module half_add (  
    input a,b,  
    output sum,carry  
);  
    assign sum = a ^ b;  
    assign carry = a & b;  
endmodule
```



A Step-by-Step Walkthrough in Nexys Video

2. Behavioral Simulation



A Step-by-Step Walkthrough in Nexys Video

3. Project Device (Board Selection)

Project device:	 Nexys Video (xc7a200tsbg484-1) 
Target language:	Verilog 
Default library:	xil_defaultlib 
Top module name:	half_add  

A Step-by-Step Walkthrough in Nexys Video

4. Constraints (Xilinx Design Constraints)

Switches

```
set_property -dict { PACKAGE_PIN E22  IOSTANDARD LVCMOS12 } [get_ports { a }]; #sw[0]
```

```
set_property -dict { PACKAGE_PIN F21  IOSTANDARD LVCMOS12 } [get_ports { b }]; #sw[1]
```

LEDs

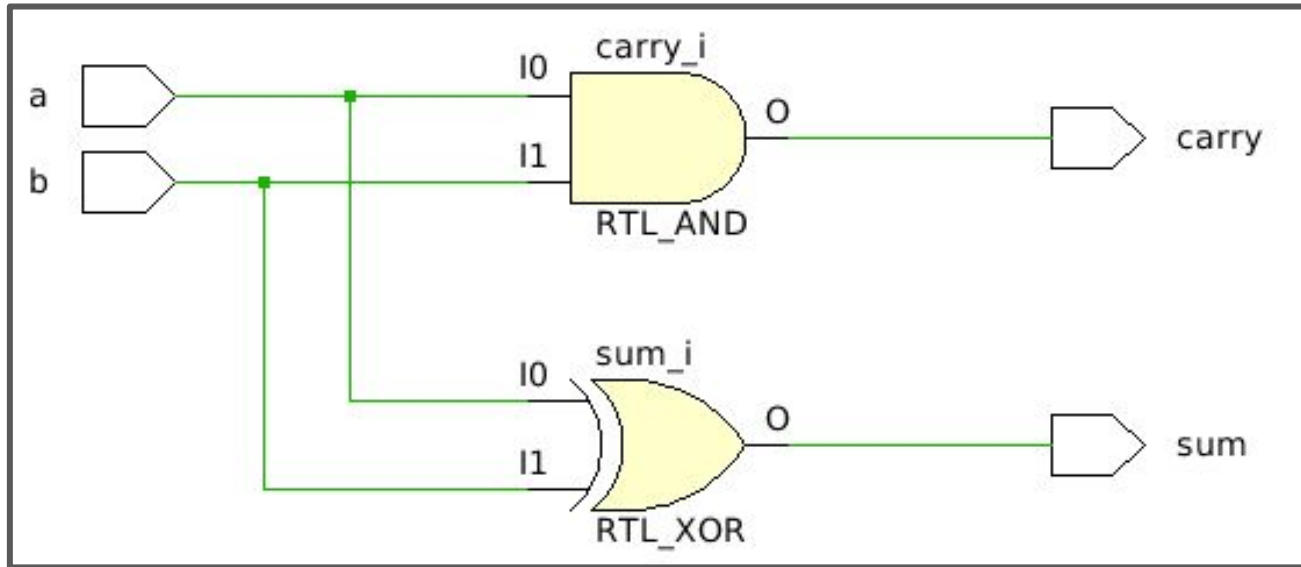
```
set_property -dict { PACKAGE_PIN T14  IOSTANDARD LVCMOS25 } [get_ports { sum }]; #led[0]
```

```
set_property -dict { PACKAGE_PIN T15  IOSTANDARD LVCMOS25 } [get_ports { carry }]; #led[1]
```

Inputs are mapped to **switches** & **Outputs** are mapped to **LEDs**.

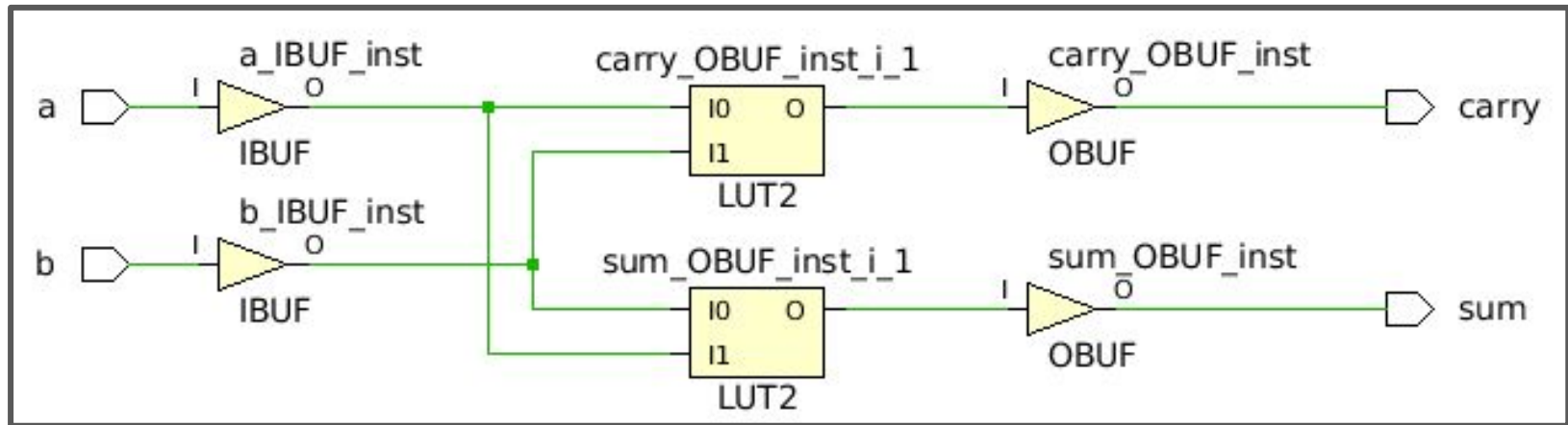
A Step-by-Step Walkthrough in Nexys Video

5. Synthesis (Translation)



A Step-by-Step Walkthrough in Nexys Video

6. Place & Route (Implementation)



Each **LUT** will have **6 inputs** in **Nexys Video**.

A Step-by-Step Walkthrough in Nexys Video

LUT Utilization

Utilization		Post-Synthesis	Post-Implementation	
			Graph	Table
Resource	Utilization	Available	Utilization %	
LUT	1	133800	0.01	
IO	4	285	1.40	



Shakti SDK



Outline

- Shakti SDK Features
- Shakti SDK Architecture
- Shakti Tools
- Drivers for Peripherals
- Pinmux
- OS
- Manuals

Shakti SDK Features

- Open-source software development platform for SHAKTI.
- Clean separation between boot, drivers, core and application layers.
- Driver support for PWM, QSPI, SPI, PLIC, CLINT, UART, I2C, GPIO, RTC, Watchdog, GPTimer and XADC.
- Multiple sensors connected and proven with SHAKTI-SDK.
- Standalone and Debug mode supported.

Shakti SDK Features

- Multilevel logging, Flash programming & Dynamic memory management supported.
- Single place for bare-metal application development, projects and benchmarks.
- SDK extendable for any FPGA board.
- SDK can be adapted to RTOS.
- Visual Studio Code IDE
 - Using Platform IO Extension

- RTOS
 - Free RTOS
 - Zephyr
 - Nuttx.
- Linux OS
- Debian Linux

Manuals

The screenshot shows a web browser window with the address bar displaying `shakti.org.in/documentation.html`. The page features a black navigation bar with the SHAKTI logo and links to Home, Processors, Software, Ideate & Build, Events, Jobs, Blog, and Forum. Below the navigation bar, the section is titled "Video Manuals". There are four video thumbnails displayed in a grid. The first two are titled "PROGRAMMING SHAKTI PART 1" and "PROGRAMMING SHAKTI PART 2", both showing a stylized background with the text "PROGRAMMING SHAKTI". The third thumbnail is titled "STEPS TO INSTALL SHAKTI SDK & SHAKTI-TOOLS" and shows hands typing on a keyboard. The fourth thumbnail is titled "STEPS TO ADD AN APPLICATION TO SHAKTI-SDK" and shows a dark background with a plus sign icon. Below each thumbnail is a caption describing the video content.

Video Manuals



PROGRAMMING SHAKTI
PART 1

Programming SHAKTI onto the FPGA boards - Part 1



PROGRAMMING SHAKTI
PART 2

Programming SHAKTI onto the FPGA boards - Part 2



STEPS TO INSTALL
SHAKTI SDK &
SHAKTI-TOOLS

Steps to install SHAKTI-SDK



STEPS TO ADD AN
APPLICATION TO
SHAKTI-SDK

Steps to add a simple application to SHAKTI-SDK

References:

Shakti Documentation: <https://shakti.org.in/documentation.html>

Shakti Blogs: <https://blogshakti.org.in/>

Shakti FPGA files: <https://gitlab.com/shaktiproject/sp2020>

Shakti SDK: <https://gitlab.com/shaktiproject/software/shakti-sdk>

Shakti on Arduino:

<https://blogshakti.org.in/how-to-print-hello-world-on-shakti-using-arduino-ide/>

Linux on Shakti: <https://gitlab.com/shaktiproject/software/linux-on-shakti>

Platform IO: <https://registry.platformio.org/platforms/platformio/shakti>

Shakti cores: <https://gitlab.com/shaktiproject/cores>

Shakti peripherals: <https://gitlab.com/shaktiproject/uncore/devices>

Queries



Thank You !

